

PRACTICAL CIE-1

QUESTION-1(B)

Solution:

```
#include <iostream>
using namespace std;
```

```
struct Team
```

```
{
```

```
    int id;
```

```
    int score;
```

```
    int rank;
```

```
};
```

```
void sortRank(Team a[], int n)
```

```
{
```

```
    int i, j;
```

```
    Team temp;
```

```
    for(i = 0; i < n - 1; i++)
```

```
    {
```

```
for(j = 0; j < n - i - 1; j++)  
{  
    if(a[j].score < a[j + 1].score ||  
        (a[j].score == a[j + 1].score &&  
         a[j].id > a[j + 1].id))  
    {  
        temp = a[j];  
        a[j] = a[j + 1];  
        a[j + 1] = temp;  
    }  
}  
}
```

```
for(i = 0; i < n; i++)  
    a[i].rank = i + 1;  
}
```

```
void sortID(Team a[], int n)  
{  
    int i, j;  
    Team temp;  
  
    for(i = 0; i < n - 1; i++)  
    {
```

```
    for(j = 0; j < n - i - 1; j++)
    {
        if(a[j].id > a[j + 1].id)
        {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}
```

```
int linearSearch(Team a[], int n, int id, int *count)
{
    int i;

    *count = 0;

    for(i = 0; i < n; i++)
    {
        (*count)++;

        if(a[i].id == id)
            return i;
    }
}
```

```
}

return -1;
}

int binarySearch(Team a[], int n, int id, int *count)
{
    int low = 0;
    int high = n - 1;
    int mid;

    *count = 0;

    while(low <= high)
    {
        mid = (low + high) / 2;

        (*count)++;

        if(a[mid].id == id)
            return mid;

        if(id < a[mid].id)
            high = mid - 1;
```

```
        else
            low = mid + 1;
    }

    return -1;
}

int main()
{
    Team teams[100], search[100];

    int n, q;
    int i, j;
    int id;
    int pos1, pos2;
    int total1 = 0, total2 = 0;
    int c1, c2;

    cout << "Enter number of teams: ";
    cin >> n;

    if(n <= 0 || n > 100)
    {
        cout << "Invalid number of teams.\n";
    }
}
```

```
        return 0;
    }

    cout << "Enter Team_ID and Score for each team:\n";

    for(i = 0; i < n; i++)
    {
        cin >> teams[i].id >> teams[i].score;
        search[i] = teams[i];
    }

    /* Create ranking */
    sortRank(teams, n);

    cout << "\nRanked List:\n";

    for(i = 0; i < n; i++)
    {
        cout << "Rank " << teams[i].rank
            << ": Team_ID " << teams[i].id
            << ", Score " << teams[i].score << endl;
    }

    cout << "\nRepeated Scores:\n";
```

```
bool found = false;
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    int count = 1;
```

```
    for(j = i + 1; j < n; j++)
```

```
    {
```

```
        if(teams[i].score == teams[j].score)
```

```
            count++;
```

```
    }
```

```
    if(count > 1)
```

```
    {
```

```
        cout << "Score " << teams[i].score
```

```
            << " -> " << count << " teams" << endl;
```

```
        found = true;
```

```
    }
```

```
    while(i + 1 < n &&
```

```
        teams[i].score == teams[i + 1].score)
```

```
    {
```

```
        i++;
```

```
    }  
}  
  
if(!found)  
    cout << "No repeated scores.\n";  
  
sortID(search, n);  
  
cout << "\nEnter number of queries: ";  
cin >> q;  
  
for(i = 0; i < q; i++)  
{  
    cout << "\nEnter Team_ID: ";  
    cin >> id;  
  
    pos1 = linearSearch(teams, n, id, &c1);  
    pos2 = binarySearch(search, n, id, &c2);  
  
    total1 += c1;  
    total2 += c2;  
  
    if(pos1 != -1)  
    {
```

```
    cout << "Team_ID: " << id << endl;
    cout << "Score: " << teams[pos1].score << endl;
    cout << "Final Rank: " << teams[pos1].rank << endl;
}
else
{
    cout << "Team_ID " << id << " not found.\n";
}

cout << "Linear Search Comparisons: "
    << c1 << endl;

cout << "Binary Search Comparisons: "
    << c2 << endl;
}

cout << "\nTotal comparisons in Linear Search: "
    << total1 << endl;

cout << "Total comparisons in Binary Search: "
    << total2 << endl;

if(total1 < total2)
    cout << "Linear Search is better.\n";
```

```
else if(total1 > total2)
    cout << "Binary Search is better.\n";
else
    cout << "Both are equally efficient.\n";

return 0;
}
```

Output:

```
Enter number of teams:3
Enter Team_ID and Score for each team:
123 56
456 67
789 79

Ranked List:
Rank 1: Team_ID 789, Score 79
Rank 2: Team_ID 456, Score 67
Rank 3: Team_ID 123, Score 56

Repeated Scores:

Enter number of queries: 2

Enter Team_ID: 123
Team_ID: 123
Score: 56
Final Rank: 3

Enter Team_ID: 457
Team_ID 457 not found.

Total comparisons in linear search: 1
Total comparisons in binary search: 4

Linear search is more efficient.
```

QUESTION-2(B)

Solution:

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int ticketID;
```

```
    int priority;
```

```
    Node* next;
```

```
    Node* prev;
```

```
};
```

```
Node* head = nullptr;
```

```
Node* tail = nullptr;
```

```
Node* current = nullptr;
```

```
int ticketCount = 0;
```

```
long long totalLinkModifications = 0;
```

```
Node* findTicket(int id) {
```

```
    Node* temp = head;
```

```
    while (temp != nullptr) {
```

```
        if (temp->ticketID == id) {
```

```
        return temp;
    }
    temp = temp->next;
}
return nullptr;
}

void newTicket(int id, int priority) {
    int linkModifications = 0;
    if (priority < 1 || priority > 5) {
        cout << "Invalid priority. Priority must be 1 to 5.\n";
        return;
    }
    if (findTicket(id) != nullptr) {
        cout << "Ticket ID already exists.\n";
        return;
    }
    Node* newNode = new Node;
    newNode->ticketID = id;
    newNode->priority = priority;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    if (head == nullptr) {
        head = newNode;
        tail = newNode;
    }
}
```

```
    current = newNode;
    ticketCount++;
}
else if (priority > head->priority) {
    newNode->next = head;
    linkModifications++;
    head->prev = newNode;
    linkModifications++;
    head = newNode;
    ticketCount++;
}
else {
    Node* temp = head;
    while (temp->next != nullptr &&
           temp->next->priority >= priority) {

        temp = temp->next;
    }
    Node* nextNode = temp->next;
    newNode->prev = temp;
    linkModifications++;
    newNode->next = nextNode;
    linkModifications++;
    temp->next = newNode;
```

```
linkModifications++;
if (nextNode != nullptr) {
    nextNode->prev = newNode;
    linkModifications++;
}
else {
    tail = newNode;
}
ticketCount++;
}
totalLinkModifications += linkModifications;
cout << "Ticket " << id << " added.\n";
cout << "Link modifications: "
    << linkModifications << endl;
}

void nextTicket() {
    if (current == nullptr) {
        cout << "No active tickets.\n";
        return;
    }
    if (current->next == nullptr) {
        cout << "Already at the last ticket.\n";
        return;
    }
}
```

```
    current = current->next;
    cout << "Current ticket: "
        << current->ticketID << endl;
}

void previousTicket() {
    if (current == nullptr) {
        cout << "No active tickets.\n";
        return;
    }
    if (current->prev == nullptr) {
        cout << "Already at the first ticket.\n";
        return;
    }
    current = current->prev;
    cout << "Current ticket: "
        << current->ticketID << endl;
}

void resolveTicket(int id) {
    int linkModifications = 0;
    Node* target = findTicket(id);
    if (target == nullptr) {
        cout << "Ticket Not Found.\n";
        return;
    }
}
```

```
if (target == head && target == tail) {
    head = nullptr;
    tail = nullptr;
    current = nullptr;
    ticketCount--;
}
else if (target == head) {
    head = target->next;
    head->prev = nullptr;
    linkModifications++;
    if (current == target) {
        current = head;
    }
    ticketCount--;
}
else if (target == tail) {
    tail = target->prev;
    tail->next = nullptr;
    linkModifications++;
    if (current == target) {
        current = tail;
    }
    ticketCount--;
}
```

```
else {
    Node* previousNode = target->prev;
    Node* nextNode = target->next;
    previousNode->next = nextNode;
    linkModifications++;
    nextNode->prev = previousNode;
    linkModifications++;
    if (current == target) {
        current = nextNode;
    }
    ticketCount--;
}
delete target;
totalLinkModifications += linkModifications;
cout << "Ticket " << id << " resolved.\n";
cout << "Link modifications: "
    << linkModifications << endl;
}

void changePriority(int id, int newPriority) {
    int linkModifications = 0;
    if (newPriority < 1 || newPriority > 5) {
        cout << "Invalid priority. Priority must be 1 to 5.\n";
        return;
    }
}
```

```
Node* target = findTicket(id);
if (target == nullptr) {
    cout << "Ticket Not Found.\n";
    return;
}
if (target->priority == newPriority) {
    cout << "Priority unchanged.\n";
    cout << "Link modifications: 0\n";
    return;
}
bool wasCurrent = (current == target);
if (target == head && target == tail) {
    head = nullptr;
    tail = nullptr;
}
else if (target == head) {
    head = target->next;
    head->prev = nullptr;
    linkModifications++;
}
else if (target == tail) {
    tail = target->prev;
    tail->next = nullptr;
    linkModifications++;
}
```

```
}  
else {  
    Node* previousNode = target->prev;  
    Node* nextNode = target->next;  
    previousNode->next = nextNode;  
    linkModifications++;  
    nextNode->prev = previousNode;  
    linkModifications++;  
}  
target->priority = newPriority;  
target->next = nullptr;  
target->prev = nullptr;  
if (head == nullptr) {  
    head = target;  
    tail = target;  
}  
else if (newPriority > head->priority) {  
    target->next = head;  
    linkModifications++;  
    head->prev = target;  
    linkModifications++;  
    head = target;  
}  
else {
```

```
Node* temp = head;

while (temp->next != nullptr &&
      temp->next->priority >= newPriority) {

    temp = temp->next;
}

Node* nextNode = temp->next;
target->prev = temp;
linkModifications++;
target->next = nextNode;
linkModifications++;
temp->next = target;
linkModifications++;
if (nextNode != nullptr) {
    nextNode->prev = target;
    linkModifications++;
}
else {
    tail = target;
}
}

if (wasCurrent) {
```

```
        current = target;
    }
    totalLinkModifications += linkModifications;
    cout << "Priority changed for Ticket "
        << id << ".\n";
    cout << "New Priority: "
        << newPriority << endl;
    cout << "Link modifications: "
        << linkModifications << endl;
}

void show() {
    if (head == nullptr) {
        cout << "No active tickets.\n";
        return;
    }
    cout << "\nActive Tickets:\n";
    Node* temp = head;
    while (temp != nullptr) {
        cout << "Ticket ID: "
            << temp->ticketID
            << " Priority: "
            << temp->priority << endl;
        temp = temp->next;
    }
}
```

```
}  
  
void showReverse() {  
    if (tail == nullptr) {  
        cout << "No active tickets.\n";  
        return;  
    }  
  
    cout << "\nTickets in Reverse Order:\n";  
    Node* temp = tail;  
    while (temp != nullptr) {  
        cout << "Ticket ID: "  
            << temp->ticketID  
            << " Priority: "  
            << temp->priority << endl;  
        temp = temp->prev;  
    }  
}  
  
void showCurrent() {  
    if (current == nullptr) {  
        cout << "No ticket is currently selected.\n";  
        return;  
    }  
  
    cout << "Current Ticket ID: "  
        << current->ticketID << endl;  
    cout << "Priority: "
```

```
        << current->priority << endl;
    }

int main() {
    string command;
    cout << "TECHNICAL SUPPORT TICKET NAVIGATOR:\n";
    cout << "\nCommands:\n";
    cout << "NEW id priority\n";
    cout << "NEXT\n";
    cout << "PREVIOUS\n";
    cout << "RESOLVE id\n";
    cout << "CHANGE id newPriority\n";
    cout << "SHOW\n";
    cout << "SHOW_REVERSE\n";
    cout << "CURRENT\n";
    cout << "EXIT\n";
    while (true) {
        cout << "\nEnter command: ";
        cin >> command;
        if (command == "NEW") {

            int id, priority;

            cin >> id >> priority;
```

```
        newTicket(id, priority);
    }
    else if (command == "NEXT") {

        nextTicket();
    }
    else if (command == "PREVIOUS") {

        previousTicket();
    }
    else if (command == "RESOLVE") {

        int id;

        cin >> id;

        resolveTicket(id);
    }
    else if (command == "CHANGE") {

        int id, priority;

        cin >> id >> priority;
```

```
        changePriority(id, priority);
    }
    else if (command == "SHOW") {

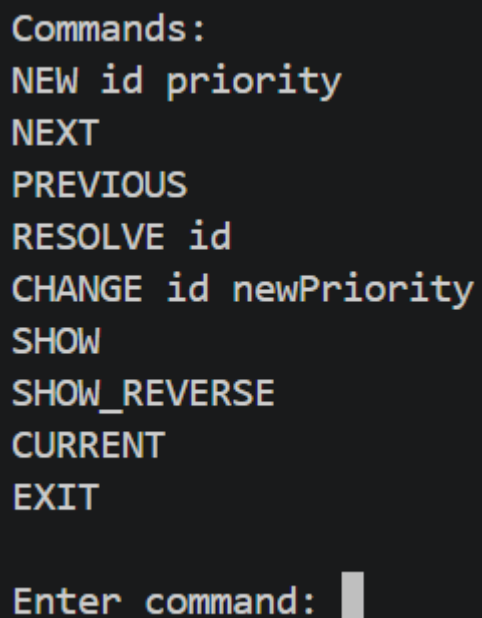
        show();
    }
    else if (command == "SHOW_REVERSE") {

        showReverse();
    }
    else if (command == "CURRENT") {
        showCurrent();
    }
    else if (command == "EXIT") {
        cout << "\nTotal link modifications: "
              << totalLinkModifications << endl;
        cout << "Program terminated.\n";
        break;
    }
    else {
        cout << "Invalid command.\n";
    }
}

Node* temp = head;
```

```
while (temp != nullptr) {  
    Node* nextNode = temp->next;  
    delete temp;  
    temp = nextNode;  
}  
return 0;  
}
```

Output:



```
Commands:  
NEW id priority  
NEXT  
PREVIOUS  
RESOLVE id  
CHANGE id newPriority  
SHOW  
SHOW_REVERSE  
CURRENT  
EXIT  
  
Enter command: █
```